Lab Assignment -19.3

Name : K . Laxmi Prasanna

Batch : 42

Rollno : 2303A52310

Task 1 – Python to C++ Conversion Task:
Translate a simple Python class into C++ using AI assistance.
Instructions:

- Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
- Ensure proper handling of:

```
class Student:
   def _init_(self, name, age):
       self.name = name self.age
       = age
 def display(self):
       print(self.name, self.age)


s = Student("John", 20)
s.display()
```

o Constructors o
Data types o
Access specifiers
• Compile and run both versions to verify output consistency
Prompt
Convert a Python class Student with attributes name, age and a display() method into equivalent C++ code with constructor and proper access specifiers. Python code

C++ code
Output:

Justification

```
/student_demo.cpp -o /tmp/student_demo
/tmp/student_demo
John 20
```

```
#include   <iostream>
using namespace std;

 class Student
{ private:
```

```
   string name; int
   age;

public:
   Student(string n, int a) {
       name = n; age = a;
   }
 void display() { cout << name << " " <<
   age << endl; }
};


int main() {
   Student s("John", 20);
   s.display(); return 0;
}
```

Justification

Constructor added explicitly in C++

Data types defined (`string`, `int`)

Encapsulation via `private/public`

Output verified same in both languages **Task 2 – Java to Python Function Conversion** Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Prompt:

Convert Java method isPrime(int n) into Python function using Pythonic syntax. Java code

Python code

Output:

Justification

Python uses indentation instead of braces

`range()` replaces loop structure

Cleaner syntax (no type declaration)

```
public static boolean isPrime(int n) {
   if(n  <=  1)  return  false;  for(int
   i=2; i<=n/2; i++) { if(n % i == 0)
   return false;
   } return
   true;
}
```

```
def is_prime(n):
   if n <= 1:
       return False
   for i in range(2, n//2 + 1):
       if n % i == 0:
           return False
   return True
 print(is_prime(7))
print(is_prime(10))
```

Logic preserved exactly

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

```
tcoding/pythonpractice$ /usr/bin/python3 /home/shashidhar/Downloads/aiassitantcoding-20260
312T084629Z-3-001/aiassitantcoding/pythonpractice/prime.py
True
False
```

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Prompt

Convert bubble sort pseudocode into Python program.

Code

Output:

Justification:

Pseudocode loops directly mapped to Python loops

Swap done using tuple unpacking

```
def bubble_sort(arr):
   n = len(arr) for i
   in range(n):
       for j in range(0, n-i-1):
           if arr[j] > arr[j+1]:
               arr[j], arr[j+1] = arr[j+1], arr[j]
   return arr
 data = [5, 2, 9, 1]
print(bubble_sort(data))
```

Algorithm correctness verified

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

```
● tcoding/pythonpractice$ /usr/bin/python3 /home/shashidhar/Downloads/aiassitantcoding-20260
  312T084629Z-3-001/aiassitantcoding/pythonpractice/bubble.py
  [1, 2, 5, 9]
```

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame Prompt:

Convert SQL query SELECT name, salary FROM employees WHERE salary > 50000 into Pandas code.

Code

Sql query

```
ELECT  name,  salary
FROM employees WHERE
salary > 50000;
```

Python pandas code

```
df = pd.DataFrame(data)

result = df[df["salary"] > 50000][["name", "salary"]] print(result)
```

Output:

```
import pandas as pd
 data =
{
   "name": ["A", "B", "C"],
   "salary": [40000, 60000, 70000]
}
```

Justification

WHERE → Boolean filtering
SELECT → column selection
Equivalent data subset returned

```
tcoding/pythonpractice$ /home/shashidhar/Downloads/aiassitantcoding-20260312T084629Z-3-001
/.venv/bin/python /home/shashidhar/Downloads/aiassitantcoding-20260312T084629Z-3-001/aiass
itantcoding/pandas_demo.py
  name  salary
1    B   60000
2    C   70000
```

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

• Converting a Python searching algorithm into C • Translating a Java sorting program into JavaScript

Instructions:

• Use AI to translate a selected algorithm between two programming languages

• Execute and test both versions

• Document translation challenges such as: o Syntax differences o Library support o Memory management

Prompt

Convert Python linear search algorithm into C language with proper memory handling.

Code

C code

```
#include <stdio.h>
```

Output:

```
def search(arr, key):
 for i in range(len(arr)):

       if arr[i] == key:

           return i

   return -1



print(search([1,2,3,4], 3))
```

```c
int search(int arr[], int n, int key) {

   for(int  i  =  0;  i  <  n;  i++)  {

   if(arr[i] == key) return i;

   }
 return -1;

}




int main() {
 int arr[] = {1,2,3,4};
 int  result  =  search(arr,  4,  3);
   printf("%d\n", result);

   return 0;

}
```

```
ple.c -o sample && ./sample
```

Justification

Manual array handling in C
No dynamic typing → explicit sizes required Memory handled via static arrays
Same logic preserved